Guia de Estudo Consolidado

Projeto: Rede Social Simples em Python + SQLite

Arquitetura, decisões de design, dificuldades técnicas e material de revisão

Sumário

Sumário 2
1. Visão Geral do Projeto 4
1.1. A Analogia do Restaurante 4
1.2. Dependência Arquitetural 4
2. A Camada de Interface — main.py ("O Garçom") 5
2.1. Papel e Responsabilidade 5
2.2. Fluxo de Controle com match/case 5
2.3. Validações e Graceful Error Handling 5
2.4. Estilização e Codificação Unicode 5
3. A Camada Intermediária — rede.py ("O Balcão") 6
3.1. O Papel da Classe RedeSocial 6
3.2. Justificativa da Existência 6
4. A Camada de Persistência — database.py ("O Cozinheiro") e init_db.py 7
4.1. CREATE TABLE IF NOT EXISTS 7
4.2. Restrição UNIQUE e nomes de usuário duplicados 7
4.3. cursor e commit — conceitos de SQLite em Python 7
4.4. Bidirecionalidade das amizades 7
4.5. Dados órfãos e ordem de remoção 7
4.6. sqlite_sequence e popular_banco.py 8
4.7. A classe Usuario (usuario.py) 8
5. O Padrão if __name__ == "__main__" 9
6. Testes Automatizados — teste_sistema.py 10
6.1. not e a inversão de lógica nos testes 10
Por que a negação confirma o resultado certo? 10
7. Resumo Técnico para a Apresentação 11
7.1. Tabela de Arquivos e Responsáveis 11
7.2. Checklist de Fluxo de Dados (ex.: Cadastro) 11
8. Decisões de Projeto e Pontos Extras 12
9. Questões Dissertativas (para responder) 13
9.1. Modularização Arquitetônica 13
9.2. Integridade de Dados e Lógica Relacional 13
9.3. Escolhas de Design da Interface do Usuário 13
9.4. Resolução de Problemas no Desenvolvimento 13
9.5. Adaptação de Linguagem (C para Python) 13
10. Glossário de Termos-Chave 14

1. Visão Geral do Projeto

O projeto consiste em uma rede social simplificada, executada via terminal, projetada para gerenciar perfis de usuário e amizades. É desenvolvido em Python e utiliza um banco de dados SQLite chamado rede_social.db para o armazenamento permanente de dados.

O sistema é organizado em sete arquivos, cada um com um papel técnico específico. Essa divisão permite que falhas na persistência sejam tratadas isoladamente da lógica de interface, facilitando o debug e a evolução do código.

1.1. A Analogia do Restaurante

Para ilustrar o fluxo de dados em uma arquitetura de software em camadas, usamos a analogia de um restaurante:

- **main.py — "O Garçom":** responsável pela interface. Captura as entradas do usuário e exibe os resultados, mas não "cozinha" os dados.
- **rede.py — "O Balcão da Cozinha":** atua como Camada de Serviço (Service Layer). É o ponto de entrega que repassa os pedidos da interface para o banco de dados.
- **database.py — "O Cozinheiro":** centraliza a lógica de persistência e operações SQL. É quem realmente manipula os "ingredientes" (dados) na "despensa" (SQLite).
- **init_db.py — "A Preparação da Cozinha":** organiza o ambiente, criando as tabelas e definindo as restrições antes do serviço começar.
- **usuario.py — "A Etiqueta do Prato":** define a classe Usuario, servindo como modelo que identifica e estrutura os dados (ID, username, nome, idade).

1.2. Dependência Arquitetural

O projeto segue um fluxo de dependência fixo: main.py (a interface) chama rede.py (a camada intermediária), que por sua vez chama database.py para se comunicar com o banco de dados SQLite. Essa estrutura modular garante que a interface do usuário nunca acesse o banco de dados diretamente — e que, em tese, a interface pudesse ser substituída (por exemplo, por uma interface Web) sem alterar a lógica de acesso aos dados.

2. A Camada de Interface — main.py ("O Garçom")

2.1. Papel e Responsabilidade

main.py é o ponto de entrada (entry point) do sistema. Gerencia estritamente a interação com o usuário através de input() e print().

Fail-safe de inicialização: o arquivo chama automaticamente a função criar_banco() do init_db.py, garantindo que o sistema funcione mesmo que o banco de dados ainda não exista.

2.2. Fluxo de Controle com match/case

O projeto adota o Dispatcher Pattern através da estrutura match/case (recurso do Python 3.10+, também chamado de structural pattern matching), usado no sistema de menus para lidar com as diferentes escolhas do usuário.

Justificativa técnica: essa abordagem é considerada mais "pythonic" que múltiplos blocos if/elif. Ela organiza visualmente as sete opções do menu, facilitando a leitura do fluxo de decisão e a manutenção da interface.

2.3. Validações e Graceful Error Handling

Para evitar que o sistema trave (crash), foi implementado tratamento de erros para melhorar a experiência do usuário (UX):

- **Validação de idade:** o uso de try/except captura a exceção ValueError caso o usuário digite caracteres não numéricos, impedindo o encerramento abrupto do programa.
- **Tratamento de strings com .strip():** aplicado em campos obrigatórios como username e nome, evita que entradas compostas apenas por espaços sejam aceitas como válidas, prevenindo erros lógicos no loop do menu.

2.4. Estilização e Codificação Unicode

A interface utiliza caracteres Unicode de desenho de caixas (box drawing — linhas duplas e simples) para criar uma interface textual estilizada (caixa_dupla e caixa_simples).

Para evitar o erro UnicodeEncodeError no Windows (onde o CMD padrão não é UTF-8), utiliza-se o comando:

```
sys.stdout.reconfigure(encoding='utf-8')
```

Isso garante a compatibilidade da interface textual rica entre sistemas operacionais.

3. A Camada Intermediária — rede.py ("O Balcão")

3.1. O Papel da Classe RedeSocial

Este arquivo implementa uma Camada de Serviço. A classe RedeSocial funciona como um mediador: recebe chamadas da interface e as encaminha para o database.py.

3.2. Justificativa da Existência

A existência desta camada é fundamental para manter a modularização. Ao garantir que main.py nunca acesse o banco de dados diretamente, cria-se um sistema onde a interface poderia ser substituída sem que a lógica de acesso aos dados fosse alterada.

4. A Camada de Persistência — database.py ("O Cozinheiro") e init_db.py

4.1. CREATE TABLE IF NOT EXISTS

A cláusula IF NOT EXISTS permite que o banco de dados e as tabelas sejam criados apenas se ainda não existirem. Isso garante que o script de inicialização possa ser executado sempre que o programa for iniciado, sem sobrescrever ou excluir acidentalmente dados de usuários já existentes.

4.2. Restrição UNIQUE e nomes de usuário duplicados

O campo username na tabela usuarios é definido com uma restrição UNIQUE. Essa abordagem é mais robusta que uma verificação simples no cliente, pois previne race conditions (condições de corrida).

Quando se tenta usar um nome de usuário já existente, o código em database.py captura um erro sqlite3.IntegrityError usando um bloco try/except, retornando False — em vez de permitir que o programa trave — e permitindo que o sistema informe o erro ao usuário de forma graciosa.

```
print("Username já existe. Repita o processo, por favor.")
```

4.3. cursor e commit — conceitos de SQLite em Python

conn.cursor() é um ponteiro? Não exatamente, mas a intuição não está longe. Um cursor é mais como um "operador" usado para executar comandos SQL e navegar pelos resultados — pense nele como a pessoa que vai até a estante (o banco) buscar ou guardar algo, usando a conexão (conn) como a chave da porta. Não é um ponteiro de memória como em C: é um objeto Python que sabe "conversar" com o SQLite.

.commit() é quem confirma as alterações, tirando-as do estado de "pendente". Analogia: é como escrever um documento e não apertar Ctrl+S — sem o commit(), as alterações ficam perdidas (não são persistidas no banco).

4.4. Bidirecionalidade das amizades

O sistema trata as amizades como bidirecionais, verificando ambas as combinações possíveis (Usuário A → Usuário B e Usuário B → Usuário A) antes de inserir um novo registro ou realizar uma exclusão. Isso garante que apenas uma linha seja necessária para representar uma conexão mútua entre duas pessoas.

Solução arquitetural adotada: em vez de processar e ordenar os dados no Python (como em uma função auxiliar, ex. par() / _par()), a lógica foi movida para a camada de dados. Utiliza-se uma consulta SQL com a cláusula OR para verificar as duas ordens possíveis antes de qualquer inserção — uma abordagem mais eficiente, por aproveitar o próprio motor de busca do banco de dados em vez de reimplementar essa verificação manualmente em Python.

4.5. Dados órfãos e ordem de remoção

Para evitar "dados órfãos" (registros em amizades apontando para um ID de usuário que já foi excluído), o sistema segue uma sequência obrigatória ao remover um usuário:

1. Excluir primeiro todos os registros na tabela amizades associados àquele ID de usuário.
2. Só então excluir o usuário da tabela usuarios.

Se a ordem fosse invertida, os registros de amizade ficariam apontando para um ID inexistente. Vale notar que este projeto gerencia a integridade referencial via código (manualmente), e não através do comando `PRAGMA foreign_keys = ON`, que não foi ativado explicitamente. Por isso, essa ordem manual de remoção é a única defesa real contra dados órfãos.

4.6. sqlite_sequence e popular_banco.py

O script popular_banco.py é uma ferramenta de suporte usada para inserir usuários e amizades de exemplo para demonstrações. Ele começa limpando todas as tabelas e reiniciando o contador de ID de incremento automático, para garantir um estado limpo e evitar erros decorrentes de entradas duplicadas.

```
cursor.execute("DELETE FROM sqlite_sequence WHERE name='usuarios'")
```

Esse comando diz ao SQLite: "esqueça que você já gerou IDs até 5, comece do 1 de novo na próxima inserção."

4.7. A classe Usuario (usuario.py)

A classe Usuario atua como uma representação orientada a objetos de um usuário, contendo atributos para ID, username, nome e idade. Ela permite que o restante da aplicação acesse dados pelos nomes dos atributos (ex.: user.name) em vez de manipular tuplas de dados brutos retornadas pelo banco de dados.

5. O Padrão if __name__ == "__main__"

Cada arquivo Python tem uma variável invisível chamada __name__.

- Quando você roda o arquivo diretamente (`python init_db.py`), o Python define __name__ como "__main__".
- Quando o arquivo é importado por outro (como `from init_db import criar_banco`), __name__ vira o nome do arquivo ("init_db"), não "__main__".

Esse if funciona como um filtro: "só execute essa função automaticamente se este arquivo foi o que rodou direto, não se foi só importado por outro arquivo". É por isso que main.py pode importar criar_banco do init_db.py sem que init_db.py tente rodar sua própria lógica de novo por acidente.

6. Testes Automatizados — teste_sistema.py

Os testes automatizados são executados por meio do teste_sistema.py, que realiza sete rotinas de verificação específicas. Ele utiliza uma função checar() para comparar os resultados reais com os esperados, exibindo um símbolo de verificação (✓) para sucesso ou um "X" (✗) para falha.

6.1. not e a inversão de lógica nos testes

Comparando duas formas equivalentes de testar o bloqueio de autoamizade:

```
# Do jeito recomendado (mais claro):
resultado = rede.criar_amizade(usuario_teste, usuario_teste)
checar("Bloqueio de autoamizade", not resultado)

# Alternativa (mais confusa):
resultado = not rede.criar_amizade(usuario_teste, usuario_teste)
checar("Bloqueio de autoamizade", resultado)
```

A vantagem da primeira forma é não alterar a informação original — resultado continua representando exatamente o que criar_amizade retornou. A inversão acontece só na hora da verificação (checar), deixando claro que estamos invertendo a lógica ali, sem escondê-la dentro da variável.

Por que a negação confirma o resultado certo?

O comportamento correto e desejado é que o sistema recuse criar amizade consigo mesmo. Ou seja, criar_amizade deve retornar False (a operação falhou, como esperado), e not False = True → o teste passa (✓).

Ideia-chave: "sucesso do teste" é o oposto de "sucesso da operação". Quando o objetivo do teste é confirmar que algo deveria falhar, o teste só passa se a operação de fato falhar.

7. Resumo Técnico para a Apresentação

7.1. Tabela de Arquivos e Responsáveis

Arquivo	Função Principal	Responsável
main.py	Interface, menus e captura de inputs.	Gustavo
rede.py	Camada de Serviço (ponte UI–DB).	Gustavo
database.py	Operações SQL (CRUD) e lógica de persistência.	Pedro
init_db.py	Definição de schema e criação das tabelas.	Pedro
usuario.py	Definição da classe/objeto de dados.	Vinícius
popular_banco.py	Script de carga de dados inicial (seed).	Vinícius
teste_sistema.py	Bateria de testes automatizados.	Vinícius
README.md	Documentação e guia de execução.	Vinícius

7.2. Checklist de Fluxo de Dados (ex.: Cadastro)

1. Entrada: main.py captura dados via input().
2. Sanitização: main.py aplica .strip() e valida tipos (ex.: int(idade)).
3. Encaminhamento: main.py chama o método correspondente em rede.py.
4. Processamento: rede.py solicita a execução para o database.py.
5. Persistência: database.py executa o INSERT no SQLite sob proteção de try/except.
6. Feedback: o resultado (sucesso/erro) retorna em cascata até ser exibido ao usuário.

8. Decisões de Projeto e Pontos Extras

- **Persistência com SQLite:** o uso de um banco relacional cumpre o requisito de armazenamento persistente, garantindo que os perfis não sejam perdidos ao reiniciar o software.
- **Interface textual estilizada:** optou-se por uma interface rica em Unicode em vez de uma GUI para focar na lógica algorítmica, buscando o ponto extra de estilização com código limpo e eficiente.
- **Escopo de login:** o sistema foca no gerenciamento de grafos de amizade e perfis. Sistemas de autenticação foram omitidos para evitar complexidade desnecessária, priorizando os requisitos principais do projeto.

9. Questões Dissertativas (para responder)

As respostas a seguir não foram fornecidas pelo material original — ficam como roteiro de estudo. Use a documentação do projeto e o conteúdo das seções anteriores como base.

9.1. Modularização Arquitetônica

Analise os benefícios de dividir o projeto em sete arquivos Python distintos. Explique como essa separação de responsabilidades facilita a manutenção e se alinha ao requisito de que a interface não deve interagir diretamente com o banco de dados.

Pista do material: "Em vez de concentrar todo o código em um único arquivo, o sistema é dividido em vários arquivos, cada um desempenhando o papel de um membro específico da equipe."

9.2. Integridade de Dados e Lógica Relacional

Discuta a importância das chaves estrangeiras (FOREIGN KEY) e da restrição UNIQUE no contexto desta rede social. De que maneira esses recursos de banco de dados simplificam o código da aplicação e, ao mesmo tempo, garantem a confiabilidade dos dados armazenados?

9.3. Escolhas de Design da Interface do Usuário

Os desenvolvedores optaram por uma interface baseada em terminal, estilizada com caracteres Unicode, em vez de uma interface gráfica (GUI). Avalie a justificativa para essa decisão, levando em consideração os requisitos do projeto e os "pontos extras" específicos buscados pelo grupo.

9.4. Resolução de Problemas no Desenvolvimento

Identifique as dificuldades técnicas mais significativas encontradas durante o projeto (como duplicatas bidirecionais e dados órfãos). Detalhe as soluções lógicas implementadas para superar esses obstáculos.

9.5. Adaptação de Linguagem (C para Python)

A tarefa original mencionava conceitos de C, como structs e gerenciamento manual de memória. Explique como o grupo traduziu esses requisitos para Python, comparando especificamente structs de C com classes de Python e discutindo as diferenças no gerenciamento de memória entre as duas linguagens.

10. Glossário de Termos-Chave

Termo	Definição
AUTOINCREMENT	Palavra-chave do SQLite usada para gerar automaticamente um ID inteiro único e crescente para cada nova linha em uma tabela.
Bidirecionalidade	Lógica aplicada a amizades, em que um único registro representa um vínculo mútuo entre dois usuários (se A é amigo de B, B é automaticamente amigo de A).
CREATE TABLE IF NOT EXISTS	Comando SQL que cria uma tabela apenas se ela ainda não existir no banco de dados, evitando erros durante inicializações repetidas.
FOREIGN KEY	Restrição de banco de dados que vincula uma coluna em uma tabela (amizades) à chave primária de outra tabela (usuarios), estabelecendo um relacionamento formal.
IntegrityError	Erro específico gerado pela biblioteca sqlite3 quando uma operação de banco de dados viola uma restrição, como tentar inserir um valor duplicado em uma coluna UNIQUE.
Match/Case	Recurso de correspondência de padrões estruturais (structural pattern matching) introduzido no Python 3.10, usado no sistema de menus do projeto para lidar com diferentes escolhas do usuário.
Modularização	Prática de dividir um programa em unidades funcionais separadas (arquivos), como separar a lógica do banco de dados (database.py) da lógica da interface (main.py).
Dados Órfãos	Registros que permanecem em uma tabela relacionada (como amizades) apontando para um registro principal (como um usuário) que foi excluído.
Persistência	Característica de dados que sobrevivem ao encerramento do programa que os criou; neste projeto, alcançada usando um arquivo de banco de dados SQLite.
PRAGMA foreign_keys = ON	Comando do SQLite necessário para forçar o banco a aplicar restrições de chave estrangeira; a documentação do projeto observa que isso não foi ativado explicitamente.
sqlite_sequence	Tabela interna do SQLite que rastreia o valor atual para colunas AUTOINCREMENT; é limpa neste projeto para reiniciar os IDs durante os testes.
Unicode (Box Drawing)	Caracteres especiais de desenho de caixas (linhas duplas e simples) usados para criar uma interface textual estilizada no terminal.
ValueError	Exceção do Python gerada quando uma função recebe um argumento do tipo correto, mas com valor inadequado — como tentar converter a string "abc" em um número inteiro.